

Raydium Onchain Bridge with JCK and JCKP Tokens in Solana Devnet prompt.txt

Setting Up Raydium Onchain Bridge with JCK and JCKP Tokens in Solana Devnet

Table of Contents

1. [Prerequisites](#)
2. [Environment Setup](#)
3. [Token Creation](#)
4. [Bridge Program Setup](#)
5. [Raydium Pool Creation](#)
6. [Bridge Integration with Raydium](#)
7. [Testing the Bridge](#)
8. [Security Considerations](#)
9. [Troubleshooting](#)

Prerequisites

Before starting, ensure you have the following installed:

- Node.js (v14 or later)
- Rust and Cargo
- Solana CLI (v1.10.0 or later)
- Solana devnet account with SOL
- Git

Environment Setup

1. Clone and Setup the Bridge Program

```
# Clone the bridge program repository
git clone https://github.com/your-username/solana-bridge-program.git
cd solana-bridge-program

# Install dependencies
npm install
```

```
# Build the bridge program
npm run build:bridge
npm run build:commission
npm run build:upgrade
```

2. Install Raydium SDK

```
npm install @raydium-io/raydium-sdk
```

3. Configure Environment Variables

Create a `.env` file in the project root:

```
# Solana Configuration
SOLANA_NETWORK=devnet
SOLANA_RPC_URL=https://api.devnet.solana.com

# Wallet Configuration
WALLET_PRIVATE_KEY=your_private_key_here

# Bridge Configuration
BRIDGE_PROGRAM_ID=your_deployed_bridge_program_id
COMMISSION_PROGRAM_ID=your_deployed_commission_program_id

# Token Configuration
JCK_TOKEN_MINT=
JCKP_TOKEN_MINT=
```

Token Creation

1. Create JCK and JCKP Tokens

Create a new file `create-tokens.js`:

```
const { Connection, Keypair, PublicKey, sendAndConfirmTransaction,
Transaction } = require('@solana/web3.js');
const { Token, TOKEN_PROGRAM_ID } = require('@solana/spl-token');
require('dotenv').config();

async function createToken(connection, payer, name, symbol, decimals) {
  // Create new token mint
  const mintAuthority = payer;
  const freezeAuthority = payer;
  const mintKeypair = Keypair.generate();
```

```

console.log(`Creating token: ${name} (${symbol})`);
console.log(`Mint address: ${mintKeypair.publicKey.toString()}`);

// Create token
const token = await Token.createMint(
  connection,
  payer,
  mintAuthority.publicKey,
  freezeAuthority.publicKey,
  decimals,
  TOKEN_PROGRAM_ID
);

// Create associated token account for the payer
const tokenAccount = await
token.getOrCreateAssociatedAccountInfo(payer.publicKey);

// Mint some tokens to the payer
await token.mintTo(
  tokenAccount.address,
  mintAuthority,
  [],
  1000000 * Math.pow(10, decimals)
);

console.log(`Minted 1,000,000 ${symbol} to
${payer.publicKey.toString()}`);

return {
  mint: token.publicKey.toString(),
  tokenAccount: tokenAccount.address.toString()
};
}

async function main() {
  // Connect to the Solana devnet
  const connection = new Connection(process.env.SOLANA_RPC_URL,
'confirmed');

  // Load the payer wallet
  const privateKeyArray = Buffer.from(process.env.WALLET_PRIVATE_KEY,
'base64');
  const payer = Keypair.fromSecretKey(privateKeyArray);

```

```

// Create JCK token
const jckToken = await createToken(connection, payer, 'JCK Token', 'JCK',
9);

// Create JCKP token
const jckpToken = await createToken(connection, payer, 'JCKP Token',
'JCKP', 9);

console.log(`\nUpdate your .env file with these values:`);
console.log(`JCK_TOKEN_MINT=${jckToken.mint}`);
console.log(`JCKP_TOKEN_MINT=${jckpToken.mint}`);
}

main().catch(console.error);

```

Run the script to create the tokens:

```
node create-tokens.js
```

2. Set Up Metadata for Tokens (Optional)

If you want to add metadata to your tokens, create a new file `add-metadata.js`:

```

const { Connection, Keypair, PublicKey, sendAndConfirmTransaction,
Transaction } = require('@solana/web3.js');
const { Token, TOKEN_PROGRAM_ID } = require('@solana/spl-token');
const { createCreateMetadataAccountV2Instruction } = require('@metaplex-
foundation/mpl-token-metadata');
require('dotenv').config();

async function addMetadata(connection, payer, mintAddress, name, symbol,
uri) {
  const mintPublicKey = new PublicKey(mintAddress);

  // Find metadata account address
  const [metadataAccount] = await PublicKey.findProgramAddress(
    [
      Buffer.from('metadata'),
      TOKEN_METADATA_PROGRAM_ID.toBuffer(),
      mintPublicKey.toBuffer(),
    ],
    TOKEN_METADATA_PROGRAM_ID
  );

```

```

// Create metadata instruction
const instruction = createCreateMetadataAccountV2Instruction(
  {
    metadata: metadataAccount,
    mint: mintPublicKey,
    mintAuthority: payer.publicKey,
    payer: payer.publicKey,
    updateAuthority: payer.publicKey,
  },
  {
    createMetadataAccountArgsV2: {
      data: {
        name,
        symbol,
        uri,
        sellerFeeBasisPoints: 0,
        creators: null,
        collection: null,
        uses: null,
      },
      isMutable: true,
    },
  }
);

// Create and send transaction
const transaction = new Transaction().add(instruction);
await sendAndConfirmTransaction(connection, transaction, [payer]);

console.log(`Added metadata to ${symbol} token`);
}

async function main() {
  // Connect to the Solana devnet
  const connection = new Connection(process.env.SOLANA_RPC_URL,
'confirmed');

  // Load the payer wallet
  const privateKeyArray = Buffer.from(process.env.WALLET_PRIVATE_KEY,
'base64');
  const payer = Keypair.fromSecretKey(privateKeyArray);

  // Add metadata to JCK token

```

```

await addMetadata(
  connection,
  payer,
  process.env.JCK_TOKEN_MINT,
  'JCK Token',
  'JCK',
  'https://your-metadata-uri.com/jck.json'
);

// Add metadata to JCKP token
await addMetadata(
  connection,
  payer,
  process.env.JCKP_TOKEN_MINT,
  'JCKP Token',
  'JCKP',
  'https://your-metadata-uri.com/jckp.json'
);
}

main().catch(console.error);

```

Bridge Program Setup

1. Deploy the Bridge Programs

```

# Deploy the bridge program
solana program deploy dist/program/bridge.so

# Deploy the commission program
solana program deploy dist/program/commission.so

# Deploy the upgrade program
solana program deploy dist/program/upgrade.so

# Update your .env file with the program IDs

```

2. Initialize the Bridge Admin

Create a new file `initialize-bridge.js`:

```

const { Connection, Keypair, PublicKey, sendAndConfirmTransaction,
Transaction } = require('@solana/web3.js');
const { ec: EC } = require('elliptic');
const crypto = require('crypto');

```

```

require('dotenv').config();

// Initialize elliptic curve for ECDSA
const ec = new EC('secp256k1');

async function initializeBridge(connection, payer, bridgeProgramId,
commissionProgramId) {
  // Generate ECDSA key pair for the bridge
  const ecdsaKeypair = ec.genKeyPair();
  const publicKeyBytes =
Buffer.from(ecdsaKeypair.getPublic().encodeCompressed());

  // Generate random seed for bridge admin account
  const seeds = crypto.randomBytes(32);

  // Find bridge admin address
  const [bridgeAdminAddress] = await PublicKey.findProgramAddress(
    [seeds],
    new PublicKey(bridgeProgramId)
  );

  console.log(`Bridge Admin Address: ${bridgeAdminAddress.toString()}`);
  console.log(`ECDSA Public Key: ${publicKeyBytes.toString('hex')}`);
  console.log(`ECDSA Private Key (SAVE THIS SECURELY):
${ecdsaKeypair.getPrivate().toString('hex')}`);

  // Create initialize bridge admin instruction
  const initializeAdminInstruction = {
    programId: new PublicKey(bridgeProgramId),
    keys: [
      { pubkey: bridgeAdminAddress, isSigner: false, isWritable: true },
      { pubkey: payer.publicKey, isSigner: true, isWritable: true },
      { pubkey: SystemProgram.programId, isSigner: false, isWritable: false
},
      { pubkey: SYSVAR_RENT_PUBKEY, isSigner: false, isWritable: false },
    ],
    data: Buffer.from([
      0, // instruction index for InitializeAdmin
      ...publicKeyBytes,
      ...seeds,
      ...new PublicKey(commissionProgramId).toBuffer(),
    ]),
  };

```

```

// Create and send transaction
const transaction = new Transaction().add(initializeAdminInstruction);
await sendAndConfirmTransaction(connection, transaction, [payer]);

console.log('Bridge admin initialized successfully');

return {
  bridgeAdminAddress: bridgeAdminAddress.toString(),
  ecdsaPublicKey: publicKeyBytes.toString('hex'),
  ecdsaPrivateKey: ecdsaKeypair.getPrivate().toString('hex'),
  seeds: Buffer.from(seeds).toString('hex'),
};
}

async function main() {
  // Connect to the Solana devnet
  const connection = new Connection(process.env.SOLANA_RPC_URL,
'confirmed');

  // Load the payer wallet
  const privateKeyArray = Buffer.from(process.env.WALLET_PRIVATE_KEY,
'base64');
  const payer = Keypair.fromSecretKey(privateKeyArray);

  // Initialize the bridge
  const bridgeInfo = await initializeBridge(
    connection,
    payer,
    process.env.BRIDGE_PROGRAM_ID,
    process.env.COMMISSION_PROGRAM_ID
  );

  console.log('\nUpdate your .env file with these values:');
  console.log(`BRIDGE_ADMIN_ADDRESS=${bridgeInfo.bridgeAdminAddress}`);
  console.log(`BRIDGE_ECDSA_PUBLIC_KEY=${bridgeInfo.ecdsaPublicKey}`);
  console.log(`BRIDGE_ECDSA_PRIVATE_KEY=${bridgeInfo.ecdsaPrivateKey}`);
  console.log(`BRIDGE_SEEDS=${bridgeInfo.seeds}`);
}

main().catch(console.error);

```

Run the script to initialize the bridge:


```
node initialize-bridge.js
```

Raydium Pool Creation

1. Install Required Dependencies

```
npm install @raydium-io/raydium-sdk @solana/spl-token
```

2. Create Raydium Pools for JCK and JCKP Tokens

Create a new file `create-raydium-pools.js`:

```
const { Connection, Keypair, PublicKey } = require('@solana/web3.js');
const { Token, TOKEN_PROGRAM_ID } = require('@solana/spl-token');
const { Liquidity, TokenAmount, Percent, Token: SDKToken, Currency } =
require('@raydium-io/raydium-sdk');
require('dotenv').config();

async function createRaydiumPool(
  connection,
  payer,
  tokenMintAddress,
  tokenSymbol,
  baseAmount,
  quoteAmount
) {
  // Connect to token mints
  const tokenMint = new PublicKey(tokenMintAddress);
  const wsolMint = new
PublicKey('So11111111111111111111111111111111111111112'); // Wrapped SOL

  // Setup token accounts
  const baseToken = new SDKToken(connection, tokenMint, 9, tokenSymbol,
tokenSymbol);
  const quoteToken = new SDKToken(connection, wsolMint, 9, 'WSOL', 'Wrapped
SOL');

  // Initialize Raydium Liquidity provider
  const liquidity = new Liquidity(
    connection,
    Liquidity.makeAmmId(baseToken, quoteToken),
    payer
  );

  // Get token accounts
```

```

const baseTokenAccount = await Token.getAssociatedTokenAddress(
  TOKEN_PROGRAM_ID,
  payer.publicKey,
  tokenMint
);

const wsolTokenAccount = await Token.getAssociatedTokenAddress(
  TOKEN_PROGRAM_ID,
  payer.publicKey,
  wsolMint
);

// Create pool
const poolInfo = await liquidity.createPool({
  baseToken,
  quoteToken,
  baseAmount: new TokenAmount(baseToken, baseAmount),
  quoteAmount: new TokenAmount(quoteToken, quoteAmount),
  startPrice: quoteAmount / baseAmount, // Initial price
  slippage: new Percent(5, 100), // 5% slippage allowed
});

console.log(`Created Raydium pool for ${tokenSymbol}/WSOL`);
console.log(`Pool ID: ${poolInfo.poolId.toString()}`);
console.log(`LP Token: ${poolInfo.lpToken.toString()}`);

return {
  poolId: poolInfo.poolId.toString(),
  lpToken: poolInfo.lpToken.toString(),
  ammId: poolInfo.ammId.toString(),
};
}

async function main() {
  // Connect to the Solana devnet
  const connection = new Connection(process.env.SOLANA_RPC_URL,
'confirmed');

  // Load the payer wallet
  const privateKeyArray = Buffer.from(process.env.WALLET_PRIVATE_KEY,
'base64');

```

Solana Bridge Program for JCK and JCKP Tokens

Table of Contents

1. [Introduction](#introduction)
2. [Prerequisites](#prerequisites)
3. [Project Structure](#project-structure)
4. [Environment Setup](#environment-setup)
5. [Bridge Implementation Steps](#bridge-implementation-steps)
 - [Step 1: Smart Contract Development](#step-1-smart-contract-development)
 - [Step 2: Oracle Integration](#step-2-oracle-integration)
 - [Step 3: Liquidity Pool Setup](#step-3-liquidity-pool-setup)
 - [Step 4: Token Wrapping](#step-4-token-wrapping)
 - [Step 5: Bridge Operations](#step-5-bridge-operations)
6. [Security Considerations](#security-considerations)
7. [Testing](#testing)
8. [Deployment](#deployment)
9. [Maintenance](#maintenance)
10. [Troubleshooting](#troubleshooting)

Introduction

This guide provides comprehensive instructions for building a token bridge between JCK and JCKP tokens on Solana using Raydium SDK. The bridge enables seamless transfers between the two tokens with automatic price adjustments and liquidity management.

What is a Token Bridge?

A token bridge allows tokens to move between different blockchain networks or token standards. In this case, we're creating a bridge between two SPL tokens on Solana (JCK and JCKP), enabling users to easily convert between them at market rates.

How Our Bridge Works

1. User deposits JCK tokens into the bridge contract
2. Bridge contract swaps JCK for JCKP using Raydium liquidity pools
3. Bridge contract sends JCKP tokens to the user
4. The process works in reverse for JCKP to JCK conversions

Prerequisites

- Rust 1.70.0+ and Cargo
- Solana CLI 1.16.0+
- Node.js 16+ and npm/yarn
- Anchor Framework 0.28.0+
- Git
- JCK and JCKP token mint addresses
- Access to a Solana RPC endpoint (devnet for testing, mainnet for production)

Project Structure

Here's the recommended file structure for this project:

solana-bridge-program/

```
|— .env.example # Example environment variables
|— .gitignore # Git ignore file
|— Anchor.toml # Anchor configuration
|— Cargo.toml # Rust dependencies
|— README.md # Project documentation
|— migrations/ # Deployment scripts
|   |— deploy.ts # Main deployment script
|— programs/ # Solana programs (smart contracts)
|   |— bridge/ # Main bridge program
|   |   |— Cargo.toml # Program-specific dependencies
|   |   |— src/ # Program source code
|   |   |   |— lib.rs # Main program entry
|   |   |   |— state.rs # Program state definitions
|   |   |   |— instructions/ # Program instructions
|   |   |   |   |— mod.rs # Instructions module
|   |   |   |   |— initialize.rs # Bridge initialization
|   |   |   |   |— deposit.rs # Token deposit logic
|   |   |   |   |— withdraw.rs # Token withdrawal logic
|   |   |   |   |— swap.rs # Token swap logic
|   |   |   |   |— admin.rs # Administrative functions
|   |   |— utils/ # Utility functions
|   |   |   |— mod.rs # Utils module
|   |   |   |— raydium.rs # Raydium integration
|   |   |   |— oracle.rs # Price oracle integration
|   |   |   |— validation.rs # Input validation
|   |— errors.rs # Custom program errors
|— app/ # Frontend application
|   |— package.json # Frontend dependencies
|   |— src/ # Frontend source code
|   |   |— App.tsx # Main application component
```

- | | | — components/ # React components
- | | | | — BridgeForm.tsx # Bridge interaction form
- | | | | — TokenSelector.tsx # Token selection component
- | | | | — TransactionStatus.tsx # Transaction status display
- | | | — hooks/ # Custom React hooks
- | | | | — useBridge.ts # Bridge interaction hook
- | | | | — useWallet.ts # Wallet connection hook
- | | | — utils/ # Frontend utilities
- | | | — constants.ts # Constants and addresses
- | | | — helper.ts # Helper functions
- | | — public/ # Public assets
- | — tests/ # Test files
- | | — bridge.spec.ts # Main bridge tests
- | | — raydium-integration.spec.ts # Raydium integration tests
- | | — security.spec.ts # Security tests
- | — scripts/ # Utility scripts
- | — create-pool.ts # Script to create Raydium pool
- | — setup-oracle.ts # Script to set up price oracle
- | — monitor-bridge.ts # Bridge monitoring script

Environment Setup

1. Install Required Tools

```
```bash
```

```
Install Rust
```

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

```
source $HOME/.cargo/env
```

```
Install Solana CLI
```

```
sh -c "$(curl -sSfL https://release.solana.com/v1.16.0/install)"
```

```
export PATH="$HOME/.local/share/solana/install/active_release/bin:$PATH"
```

```
Install Anchor
```

```
npm install -g @coral-xyz/anchor-cli
```

```
Verify installations
```

```
rustc --version
```

```
solana --version
```

```
anchor --version
```

## 2. Configure Solana CLI for Devnet

```
solana config set --url https://api.devnet.solana.com
solana-keygen new --outfile ~/.config/solana/devnet.json
solana config set --keypair ~/.config/solana/devnet.json
solana airdrop 2 # Request devnet SOL
```

### 3. Project Setup

```
Create the project using Anchor
anchor init solana-bridge-program
cd solana-bridge-program

Create .env file
cat > .env << EOF
SOLANA_CLUSTER=devnet
SOLANA_RPC_URL=https://api.devnet.solana.com
WALLET_PATH=$HOME/.config/solana/devnet.json
JCK_TOKEN_MINT=your_jck_token_mint_address
JCKP_TOKEN_MINT=your_jckp_token_mint_address
ORACLE_UPDATE_AUTHORITY=oracle_update_authority_pubkey
ADMIN_PUBKEY=admin_pubkey
BRIDGE_FEE_BPS=30 # 0.3% fee
RAYDIUM_SWAP_PROGRAM_ID=AMMR8X2tajWmfPL9VA86hnDz7NFboMxFmYKHoinvQJtC
EOF

Install dependencies
npm install @solana/web3.js @solana/spl-token @coral-xyz/anchor @raydium-
io/raydium-sdk
```

## Bridge Implementation Steps

### Step 1: Smart Contract Development

First, we'll develop the core smart contract (Solana program) for our bridge.

#### Create Base Program Structure

Edit the `programs/bridge/src/lib.rs` file:

```
use anchor_lang::prelude::*;
use anchor_spl::token::{self, Token, TokenAccount, Mint, Transfer};
use anchor_lang::solana_program::program::{invoke, invoke_signed};

pub mod state;
pub mod instructions;
pub mod errors;
pub mod utils;
```

```

use instructions::*;
use state::*;
use errors::*;

declare_id!("your_program_id_here"); // Will be generated during deployment

#[program]
pub mod bridge {
 use super::*;

 // Initialize the bridge with configuration settings
 pub fn initialize(
 ctx: Context<Initialize>,
 fee_bps: u16,
) -> Result<()> {
 instructions::initialize::handler(ctx, fee_bps)
 }

 // Deposit JCK tokens to receive JCKP tokens
 pub fn deposit_jck(
 ctx: Context<DepositJck>,
 amount: u64,
) -> Result<()> {
 instructions::deposit::deposit_jck_handler(ctx, amount)
 }

 // Deposit JCKP tokens to receive JCK tokens
 pub fn deposit_jckp(
 ctx: Context<DepositJckp>,
 amount: u64,
) -> Result<()> {
 instructions::deposit::deposit_jckp_handler(ctx, amount)
 }

 // Execute swap via Raydium pools
 pub fn execute_swap(
 ctx: Context<ExecuteSwap>,
 min_amount_out: u64,
) -> Result<()> {
 instructions::swap::handler(ctx, min_amount_out)
 }
}

```

```

// Withdraw tokens to user
pub fn withdraw(
 ctx: Context<Withdraw>,
) -> Result<()> {
 instructions::withdraw::handler(ctx)
}

// Admin function to update bridge settings
pub fn update_settings(
 ctx: Context<UpdateSettings>,
 new_fee_bps: Option<u16>,
 new_admin: Option<Pubkey>,
) -> Result<()> {
 instructions::admin::update_settings_handler(
 ctx,
 new_fee_bps,
 new_admin
)
}

// Admin function to pause/unpause the bridge
pub fn set_pause_status(
 ctx: Context<SetPauseStatus>,
 paused: bool,
) -> Result<()> {
 instructions::admin::set_pause_status_handler(ctx, paused)
}
}

```

## Define Program State

Create `programs/bridge/src/state.rs`:

```

use anchor_lang::prelude::*;

// Main bridge configuration and state
#[account]
pub struct BridgeState {
 // Admin who can update bridge settings
 pub admin: Pubkey,
 // Fee in basis points (1/100 of 1%)
 pub fee_bps: u16,
 // Whether the bridge is paused
 pub paused: bool,
 // JCK token mint
}

```



```

pub jck_mint: Pubkey,
// JCKP token mint
pub jckp_mint: Pubkey,
// Raydium pool ID for JCK/JCKP
pub pool_id: Pubkey,
// Total JCK tokens processed
pub total_jck_processed: u64,
// Total JCKP tokens processed
pub total_jckp_processed: u64,
// Bump seed for PDA derivation
pub bump: u8,
}

// User transaction state to track in-progress transfers
#[account]
pub struct UserTransaction {
 // User wallet
 pub user: Pubkey,
 // Input token mint
 pub input_token: Pubkey,
 // Amount of input tokens
 pub input_amount: u64,
 // Output token mint
 pub output_token: Pubkey,
 // Expected minimum output amount
 pub min_output_amount: u64,
 // Status of the transaction
 pub status: TransactionStatus,
 // Timestamp of creation
 pub created_at: i64,
 // Timestamp of completion
 pub completed_at: Option<i64>,
 // Transaction bump seed
 pub bump: u8,
}

// Transaction status enum
#[derive(AnchorSerialize, AnchorDeserialize, Clone, PartialEq, Eq)]
pub enum TransactionStatus {
 // Deposit received, awaiting swap
 Deposited,
 // Swap completed, awaiting withdrawal
 Swapped,

```

```

 // Transaction completed
 Completed,
 // Transaction failed
 Failed,
}

```

## Implement Instructions

Create `programs/bridge/src/instructions/mod.rs`:

```

pub mod initialize;
pub mod deposit;
pub mod swap;
pub mod withdraw;
pub mod admin;

pub use initialize::*;
pub use deposit::*;
pub use swap::*;
pub use withdraw::*;
pub use admin::*;

```

Create `programs/bridge/src/instructions/initialize.rs`:

```

use anchor_lang::prelude::*;
use anchor_spl::token::{Mint, Token};
use crate::state::*;
use crate::errors::*;

#[derive(Accounts)]
pub struct Initialize<'info> {
 #[account(mut)]
 pub admin: Signer<'info>,

 pub jck_mint: Account<'info, Mint>,

 pub jckp_mint: Account<'info, Mint>,

 #[account(
 init,
 payer = admin,
 space = 8 + std::mem::size_of::<BridgeState>(),
 seeds = [b"bridge_state"],
 bump,
)]
}

```

```

pub bridge_state: Account<'info, BridgeState>,

#[account(
 init,
 payer = admin,
 space = 0,
 seeds = [b"bridge_authority"],
 bump,
)]
/// CHECK: This is a PDA used as token account authority
pub bridge_authority: UncheckedAccount<'info>,

pub system_program: Program<'info, System>,
pub token_program: Program<'info, Token>,
pub rent: Sysvar<'info, Rent>,
}

pub fn handler(
 ctx: Context<Initialize>,
 fee_bps: u16,
) -> Result<()> {
 // Validate fee (max 5%)
 if fee_bps > 500 {
 return Err(BridgeError::FeeTooHigh.into());
 }

 let bridge_state = &mut ctx.accounts.bridge_state;
 let bump = *ctx.bumps.get("bridge_state").unwrap();

 bridge_state.admin = ctx.accounts.admin.key();
 bridge_state.fee_bps = fee_bps;
 bridge_state.paused = false;
 bridge_state.jck_mint = ctx.accounts.jck_mint.key();
 bridge_state.jckp_mint = ctx.accounts.jckp_mint.key();
 bridge_state.total_jck_processed = 0;
 bridge_state.total_jckp_processed = 0;
 bridge_state.bump = bump;

 // Pool ID will be set later in a separate transaction
 bridge_state.pool_id = Pubkey::default();

 Ok(())
}

```

Create a few more instruction files. For brevity, I'll show just one more example, the deposit instruction:

Create `programs/bridge/src/instructions/deposit.rs`:

```
use anchor_lang::prelude::*;
use anchor_spl::token::{self, Mint, Token, TokenAccount, Transfer};
use crate::state::*;
use crate::errors::*;

#[derive(Accounts)]
pub struct DepositJck<'info> {
 #[account(mut)]
 pub user: Signer<'info>,

 #[account(
 seeds = [b"bridge_state"],
 bump = bridge_state.bump,
)]
 pub bridge_state: Account<'info, BridgeState>,

 #[account(
 mut,
 constraint = user_jck_account.mint == bridge_state.jck_mint,
 constraint = user_jck_account.owner == user.key(),
)]
 pub user_jck_account: Account<'info, TokenAccount>,

 #[account(
 mut,
 seeds = [b"bridge_jck_account"],
 bump,
)]
 pub bridge_jck_account: Account<'info, TokenAccount>,

 #[account(
 init,
 payer = user,
 space = 8 + std::mem::size_of::<UserTransaction>(),
 seeds = [b"user_transaction", user.key().as_ref(),
&bridge_state.jck_mint.to_bytes()],
 bump,
)]
 pub user_transaction: Account<'info, UserTransaction>,
```

```

pub system_program: Program<'info, System>,
pub token_program: Program<'info, Token>,
pub rent: Sysvar<'info, Rent>,
}

pub fn deposit_jck_handler(
 ctx: Context<DepositJck>,
 amount: u64,
) -> Result<()> {
 // Check if bridge is paused
 if ctx.accounts.bridge_state.paused {
 return Err(BridgeError::BridgePaused.into());
 }

 // Validate amount
 if amount == 0 {
 return Err(BridgeError::InvalidAmount.into());
 }

 // Transfer JCK tokens from user to bridge account
 let transfer_ctx = CpiContext::new(
 ctx.accounts.token_program.to_account_info(),
 Transfer {
 from: ctx.accounts.user_jck_account.to_account_info(),
 to: ctx.accounts.bridge_jck_account.to_account_info(),
 authority: ctx.accounts.user.to_account_info(),
 },
);
 token::transfer(transfer_ctx, amount)?;

 // Initialize user transaction record
 let user_transaction = &mut ctx.accounts.user_transaction;
 let bump = *ctx.bumps.get("user_transaction").unwrap();

 user_transaction.user = ctx.accounts.user.key();
 user_transaction.input_token = ctx.accounts.bridge_state.jck_mint;
 user_transaction.input_amount = amount;
 user_transaction.output_token = ctx.accounts.bridge_state.jckp_mint;
 user_transaction.min_output_amount = 0; // Will be set during swap
 user_transaction.status = TransactionStatus::Deposited;
 user_transaction.created_at = Clock::get()?.unix_timestamp;
 user_transaction.completed_at = None;
 user_transaction.bump = bump;

```

```

 // Update bridge state
 ctx.accounts.bridge_state.total_jck_processed =
ctx.accounts.bridge_state.total_jck_processed.checked_add(amount).unwrap();

 Ok(())
}

```

## Create Error Types

Create `programs/bridge/src/errors.rs`:

```

use anchor_lang::prelude::*;

#[error_code]
pub enum BridgeError {
 #[msg("Bridge is paused")]
 BridgePaused,

 #[msg("Invalid amount")]
 InvalidAmount,

 #[msg("Fee too high")]
 FeeTooHigh,

 #[msg("Insufficient funds")]
 InsufficientFunds,

 #[msg("Unauthorized")]
 Unauthorized,

 #[msg("Invalid transaction status")]
 InvalidTransactionStatus,

 #[msg("Slippage exceeded")]
 SlippageExceeded,

 #[msg("Pool not found")]
 PoolNotFound,

 #[msg("Oracle price stale")]
 OraclePriceStale,
}

```

## Step 2: Oracle Integration

Implement price oracle functionality to ensure fair exchange rates and prevent manipulation.

## Create Oracle Module

Create `programs/bridge/src/utils/oracle.rs`:

```
use anchor_lang::prelude::*;
use anchor_lang::solana_program::clock::Clock;

// Price feed account structure
#[account]
pub struct PriceFeed {
 // Oracle authority that can update prices
 pub authority: Pubkey,
 // JCK price in USD (6 decimal places)
 pub jck_price_usd: u64,
 // JCKP price in USD (6 decimal places)
 pub jckp_price_usd: u64,
 // Last updated timestamp
 pub last_updated_at: i64,
 // Maximum allowed staleness in seconds
 pub max_staleness: i64,
 // Bump seed
 pub bump: u8,
}

// Oracle instruction
#[derive(Accounts)]
pub struct UpdatePrices<'info> {
 #[account(
 mut,
 seeds = [b"price_feed"],
 bump = price_feed.bump,
)]
 pub price_feed: Account<'info, PriceFeed>,

 #[account(
 constraint = authority.key() == price_feed.authority @
BridgeError::Unauthorized
)]
 pub authority: Signer<'info>,

 pub system_program: Program<'info, System>,
}
```

```

// Function to initialize price feed
pub fn initialize_price_feed(
 ctx: Context<InitializePriceFeed>,
 jck_price_usd: u64,
 jckp_price_usd: u64,
 max_staleness: i64,
) -> Result<()> {
 let price_feed = &mut ctx.accounts.price_feed;
 let bump = *ctx.bumps.get("price_feed").unwrap();

 price_feed.authority = ctx.accounts.authority.key();
 price_feed.jck_price_usd = jck_price_usd;
 price_feed.jckp_price_usd = jckp_price_usd;
 price_feed.last_updated_at = Clock::get()?.unix_timestamp;
 price_feed.max_staleness = max_staleness;
 price_feed.bump = bump;

 Ok(())
}

```

```

// Function to update prices
pub fn update_prices(
 ctx: Context<UpdatePrices>,
 jck_price_usd: u64,
 jckp_price_usd: u64,
) -> Result<()> {
 let price_feed = &mut ctx.accounts.price_feed;

 price_feed.jck_price_usd = jck_price_usd;
 price_feed.jckp_price_usd = jckp_price_usd;
 price_feed.last_updated_at = Clock::get()?.unix_timestamp;

 Ok(())
}

```

```

// Function to get conversion rate with validation
pub fn get_conversion_rate(
 price_feed: &Account<PriceFeed>,
 from_mint: Pubkey,
 to_mint: Pubkey,
 bridge_state: &Account<BridgeState>,
) -> Result<f64> {

```



```

// Check for staleness
let current_time = Clock::get()?.unix_timestamp;
if current_time - price_feed.last_updated_at > price_feed.max_staleness
{
 return Err(BridgeError::OraclePriceStale.into());
}

// Determine conversion rate based on token direction
let rate = if from_mint == bridge_state.jck_mint && to_mint ==
bridge_state.jckp_mint {
 // JCK to JCKP: how many JCKP per 1 JCK
 price_feed.jck_price_usd as f64 / price_feed.jckp_price_usd as f64
} else if from_mint == bridge_state.jckp_mint && to_mint ==
bridge_state.jck_mint {
 // JCKP to JCK: how many JCK per 1 JCKP
 price_feed.jckp_price_usd as f64 / price_feed.jck_price_usd as f64
} else {
 return Err(BridgeError::InvalidAmount.into());
};

Ok(rate)
}

```

### Step 3: Liquidity Pool Setup

In this step, we'll integrate with Raydium to create and interact with a liquidity pool for JCK/JCKP.

#### Create Raydium Integration

Create `programs/bridge/src/utils/raydium.rs`:

```

use anchor_lang::prelude::*;
use anchor_lang::solana_program::program::invoke_signed;
use anchor_spl::token::{Token, TokenAccount, Mint};

// Constants for Raydium programs
pub const RAYDIUM_SWAP_PROGRAM_ID: &str =
"AMMR8X2tajWmfPL9VA86hnDz7NFboMxFmYKHoinvQJtC";

// Swap instruction accounts
#[derive(Accounts)]
pub struct RaydiumSwap<'info> {
 #[account(mut)]
 pub authority: Signer<'info>,

```

```

#[account(mut)]
pub user_source_token: Account<'info, TokenAccount>,

#[account(mut)]
pub user_destination_token: Account<'info, TokenAccount>,

#[account(mut)]
/// CHECK: Handled by Raydium
pub pool_program_id: UncheckedAccount<'info>,

#[account(mut)]
/// CHECK: Handled by Raydium
pub pool_id: UncheckedAccount<'info>,

#[account(mut)]
/// CHECK: Handled by Raydium
pub pool_authority: UncheckedAccount<'info>,

#[account(mut)]
/// CHECK: Handled by Raydium
pub pool_source_token: UncheckedAccount<'info>,

#[account(mut)]
/// CHECK: Handled by Raydium
pub pool_destination_token: UncheckedAccount<'info>,

#[account(mut)]
/// CHECK: Handled by Raydium
pub pool_lp_token: UncheckedAccount<'info>,

#[account(mut)]
/// CHECK: Handled by Raydium
pub pool_open_orders: UncheckedAccount<'info>,

#[account(address = anchor_lang::solana_program::sysvar::rent::ID)]
/// CHECK: Handled by Raydium
pub rent: UncheckedAccount<'info>,

pub token_program: Program<'info, Token>,
}

// Function to perform a swap through Raydium
pub fn perform_raydium_swap(

```

```

 ctx: CpiContext<'_, '_, '_, 'info, RaydiumSwap<'info>>,
 amount_in: u64,
 min_amount_out: u64,
) -> Result<()> {
 // Raydium swap instruction data
 let data = raydium_swap_instruction_data(amount_in, min_amount_out);

 // Create the accounts for the swap
 let accounts = [
 ctx.accounts.authority.to_account_info(),
 ctx.accounts.user_source_token.to_account_info(),
 ctx.accounts.user_destination_token.to_account_info(),
 ctx.accounts.pool_program_id.to_account_info(),
 ctx.accounts.pool_id.to_account_info(),
 ctx.accounts.pool_authority.to_account_info(),
 ctx.accounts.pool_source_token.to_account_info(),
 ctx.accounts.pool_destination_token.to_account_info(),
 ctx.accounts.pool_lp_token.to_account_info(),
 ctx.accounts.pool_open_orders.to_account_info(),
 ctx.accounts.token_program.to_account_info(),
 ctx.accounts.rent.to_account_info(),
];

 // Perform the swap through Raydium
 invoke_signed(
 &Instruction {
 program_id: Pubkey::from_str(RAYDIUM_SWAP_PROGRAM_ID).unwrap(),
 accounts: accounts.iter().map(|a| AccountMeta {
 pubkey: a.key(),
 is_signer: a.is_signer,
 is_writable: a.is_writable,
 }).collect(),
 data,
 },
 &accounts,
 ctx.signer_seeds,
)?;

 Ok(())
}

```

*// Helper function to create Raydium swap instruction data*

```

fn raydium_swap_instruction_data(amount_in: u64, min_amount_out: u64) ->

```

```
Vec<u8> {
 // Instruction data format: [instruction_code(u8), amount_in(u64),
 min_amount_out(u64)]
 let mut data = Vec::with_capacity(17);
 data.push(9); // Swap instruction code for Raydium
 data.extend_from_slice(&amount_in.to_le_bytes());
 data.extend_from_slice(&min_amount_out.to_le_bytes());
 data
}
```

## Step 4: Creating the Raydium Pool

We'll now set up a script to create the Raydium liquidity pool for JCK/JCKP tokens. This is a one-time operation that will be performed before the bridge can be used.

Create `scripts/create-pool.ts`:

```
import { Connection, Keypair, PublicKey } from '@solana/web3.js';
import { Liquidity, Percent, Token, TokenAmount } from '@raydium-io/raydium-
sdk';
import { readFileSync } from 'fs';
import * as dotenv from 'dotenv';

// Load environment variables
dotenv.config();

// Configure RPC and wallet
const RPC_URL = process.env.SOLANA_RPC_URL ||
'https://api.devnet.solana.com';
const WALLET_PATH = process.env.WALLET_PATH ||
'~/config/solana/devnet.json';
const JCK_MINT = process.env.JCK_TOKEN_MINT || '';
const JCKP_MINT = process.env.JCKP_TOKEN_MINT || '';

// Fee tier - 0.3% is standard
const FEE_RATE = 0.003;

// Load wallet
function loadWallet(): Keypair {
 const walletData = readFileSync(WALLET_PATH.replace('~', process.env.HOME
|| ''), 'utf-8');
 return Keypair.fromSecretKey(
 Buffer.from(JSON.parse(walletData))
);
}
```

```

}

async function createRaydiumPool() {
 console.log('Creating Raydium pool for JCK/JCKP...');

 // Initialize connection
 const connection = new Connection(RPC_URL);
 const wallet = loadWallet();

 // Get token info
 const jckMint = new PublicKey(JCK_MINT);
 const jckpMint = new PublicKey(JCKP_MINT);

 console.log(`JCK Mint: ${jckMint.toBase58()}`);
 console.log(`JCKP Mint: ${jckpMint.toBase58()}`);

 // Check wallet balance
 const balance = await connection.getBalance(wallet.publicKey);
 console.log(`Wallet balance: ${balance / 1e9} SOL`);

 if (balance < 1e9) {
 console.error('Insufficient SOL balance. Need at least 1 SOL');
 return;
 }

 // Define tokens
 const jckDecimals = 9; // Adjust based on your token
 const jckpDecimals = 9; // Adjust based on your token

 const jckToken = new Token(
 connection,
 jckMint,
 jckDecimals,
 'JCK',
 'JCK Token'
);

 const jckpToken = new Token(
 connection,
 jckpMint,
 jckpDecimals,
 'JCKP',
 'JCKP Token'
);

```

```

);

// Calculate initial liquidity amounts
// For example, 1 JCK = 100 JCKP
const baseAmount = 1000; // 1000 JCK
const quoteAmount = baseAmount * 100; // 100,000 JCKP

// Define initial liquidity
const baseTokenAmount = new TokenAmount(jckToken, `${baseAmount *
10**jckDecimals}`);
const quoteTokenAmount = new TokenAmount(jckpToken, `${quoteAmount *
10**jckpDecimals}`);

try {
 // Create pool config
 const poolConfig = {
 connection,
 wallet: {
 publicKey: wallet.publicKey,
 signTransaction: async (tx) => {
 tx.sign(wallet);
 return tx;
 },
 signAllTransactions: async (txs) => {
 txs.forEach((tx) => tx.sign(wallet));
 return txs;
 },
 },
 baseToken: jckToken,
 quoteToken: jckpToken,
 baseAmount: baseTokenAmount,
 quoteAmount: quoteTokenAmount,
 fee: new Percent(FEE_RATE * 100, 100), // 0.3%
 };

 console.log('Creating pool...');
 // Create the pool
 const poolInfo = await Liquidity.createPoolV4(poolConfig);

 console.log('Pool created successfully!');
 console.log(`Pool ID: ${poolInfo.id.toBase58()}`);

 // After creating a pool, update the bridge state with the pool ID

```

```

 console.log('Now update your bridge state with this pool ID');

} catch (error) {
 console.error('Error creating pool:', error);
}
}

// Run the script
createRaydiumPool().catch(console.error);

```

## Step 5: Bridge Operations

Now let's implement core bridge functionality including swap execution and price calculation.

Create `programs/bridge/src/instructions/swap.rs`:

```

use anchor_lang::prelude::*;
use anchor_spl::token::{Mint, Token, TokenAccount};
use crate::state::*;
use crate::errors::*;
use crate::utils::raydium::*;
use crate::utils::oracle::*;

#[derive(Accounts)]
pub struct ExecuteSwap<'info> {
 #[account(mut)]
 pub admin: Signer<'info>,

 #[account(
 mut,
 seeds = [b"bridge_state"],
 bump = bridge_state.bump,
 constraint = admin.key() == bridge_state.admin @
BridgeError::Unauthorized,
)]
 pub bridge_state: Account<'info, BridgeState>,

 #[account(
 mut,
 constraint = user_transaction.status == TransactionStatus::Deposited
@ BridgeError::InvalidTransactionStatus,
)]
 pub user_transaction: Account<'info, UserTransaction>,

```

```

// Price feed for conversion calculation
pub price_feed: Account<'info, PriceFeed>,

// Raydium pool accounts
#[account(
 mut,
 constraint = bridge_state.pool_id == pool_id.key() @
BridgeError::PoolNotFound,
)]
/// CHECK: Handled by Raydium
pub pool_id: UncheckedAccount<'info>,

#[account(mut)]
/// CHECK: Handled by Raydium
pub pool_authority: UncheckedAccount<'info>,

#[account(mut)]
pub bridge_source_token: Account<'info, TokenAccount>,

#[account(mut)]
pub bridge_destination_token: Account<'info, TokenAccount>,

// Additional Raydium accounts
#[account(mut)]
/// CHECK: Handled by Raydium
pub pool_source_token: UncheckedAccount<'info>,

#[account(mut)]
/// CHECK: Handled by Raydium
pub pool_destination_token: UncheckedAccount<'info>,

#[account(mut)]
/// CHECK: Handled by Raydium
pub pool_lp_token: UncheckedAccount<'info>,

#[account(mut)]
/// CHECK: Handled by Raydium
pub pool_open_orders: UncheckedAccount<'info>,

pub token_program: Program<'info, Token>,
pub rent: Sysvar<'info, Rent>,
}

```



```

pub fn handler(
 ctx: Context<ExecuteSwap>,
 min_amount_out: u64,
) -> Result<()> {
 // Check if bridge is paused
 if ctx.accounts.bridge_state.paused {
 return Err(BridgeError::BridgePaused.into());
 }

 let user_tx = &ctx.accounts.user_transaction;
 let input_amount = user_tx.input_amount;

 // Calculate expected output with oracle prices
 let conversion_rate = get_conversion_rate(
 &ctx.accounts.price_feed,
 user_tx.input_token,
 user_tx.output_token,
 &ctx.accounts.bridge_state
)?;

 // Calculate expected output amount (with fee deduction)
 let fee_bps = ctx.accounts.bridge_state.fee_bps as u64;
 let fee_amount =
input_amount.checked_mul(fee_bps).unwrap().checked_div(10000).unwrap();
 let adjusted_input = input_amount.checked_sub(fee_amount).unwrap();

 // Convert to output token using conversion rate
 let expected_output = (adjusted_input as f64 * conversion_rate) as u64;

 // Verify minimum output amount
 if min_amount_out > expected_output {
 // Min amount out is higher than expected, this is suspicious
 return Err(BridgeError::InvalidAmount.into());
 }

 // Apply a buffer for slippage (allow up to 1% slippage by default)
 let min_acceptable_amount =
expected_output.checked_mul(99).unwrap().checked_div(100).unwrap();

 // Ensure min_amount_out respects our minimum acceptable amount
 if min_amount_out < min_acceptable_amount {
 // Use our calculated minimum instead
 msg!("Adjusting min_amount_out to respect minimum slippage

```

```

protection");
 }

 let final_min_out = std::cmp::max(min_amount_out,
min_acceptable_amount);

 // Execute the swap through Raydium
 let raydium_accounts = RaydiumSwap {
 authority: ctx.accounts.admin.to_account_info(),
 user_source_token:
ctx.accounts.bridge_source_token.to_account_info(),
 user_destination_token:
ctx.accounts.bridge_destination_token.to_account_info(),
 pool_program_id: ctx.program_id.to_account_info(),
 pool_id: ctx.accounts.pool_id.to_account_info(),
 pool_authority: ctx.accounts.pool_authority.to_account_info(),
 pool_source_token: ctx.accounts.pool_source_token.to_account_info(),
 pool_destination_token:
ctx.accounts.pool_destination_token.to_account_info(),
 pool_lp_token: ctx.accounts.pool_lp_token.to_account_info(),
 pool_open_orders: ctx.accounts.pool_open_orders.to_account_info(),
 rent: ctx.accounts.rent.to_account_info(),
 token_program: ctx.accounts.token_program.to_account_info(),
 };

 // Create PDA signer seeds for the bridge authority
 let bridge_state_key = ctx.accounts.bridge_state.key();
 let seeds = &[b"bridge_state".as_ref(), &
[ctx.accounts.bridge_state.bump]];
 let signer_seeds = &[&seeds[..]];

 // Perform the swap
 perform_raydium_swap(
 CpiContext::new_with_signer(
 ctx.accounts.token_program.to_account_info(),
 raydium_accounts,
 signer_seeds,
),
 input_amount,
 final_min_out,
)?;

 // Update user transaction

```

```

 let user_transaction = &mut ctx.accounts.user_transaction;
 user_transaction.min_output_amount = final_min_out;
 user_transaction.status = TransactionStatus::Swapped;

 Ok(())
}

```

## Step 6: Security Setup

Implementing a secure bridge means adding proper security features and authentication. Let's create an admin module for managing bridge security:

Create `programs/bridge/src/instructions/admin.rs`:

```

use anchor_lang::prelude::*;
use crate::state::*;
use crate::errors::*;

// Account validation for updating bridge settings
#[derive(Accounts)]
pub struct UpdateSettings<'info> {
 #[account(
 constraint = admin.key() == bridge_state.admin @
 BridgeError::Unauthorized
)]
 pub admin: Signer<'info>,

 #[account(
 mut,
 seeds = [b"bridge_state"],
 bump = bridge_state.bump,
)]
 pub bridge_state: Account<'info, BridgeState>,
}

// Account validation for setting pause status
#[derive(Accounts)]
pub struct SetPauseStatus<'info> {
 #[account(
 constraint = admin.key() == bridge_state.admin @
 BridgeError::Unauthorized
)]
 pub admin: Signer<'info>,
}

```

```

#[account(
 mut,
 seeds = [b"bridge_state"],
 bump = bridge_state.bump,
)]
pub bridge_state: Account<'info, BridgeState>,
}

// Account validation for updating the pool ID
#[derive(Accounts)]
pub struct UpdatePoolId<'info> {
 #[account(
 constraint = admin.key() == bridge_state.admin @
BridgeError::Unauthorized
)]
 pub admin: Signer<'info>,

 #[account(
 mut,
 seeds = [b"bridge_state"],
 bump = bridge_state.bump,
)]
 pub bridge_state: Account<'info, BridgeState>,

 /// CHECK: Handled by program
 pub new_pool_id: UncheckedAccount<'info>,
}

// Handler for updating bridge settings
pub fn update_settings_handler(
 ctx: Context<UpdateSettings>,
 new_fee_bps: Option<u16>,
 new_admin: Option<Pubkey>,
) -> Result<()> {
 let bridge_state = &mut ctx.accounts.bridge_state;

 // Update fee if provided
 if let Some(fee_bps) = new_fee_bps {
 // Validate fee (max 5%)
 if fee_bps > 500 {
 return Err(BridgeError::FeeTooHigh.into());
 }
 bridge_state.fee_bps = fee_bps;
 }
}

```

```

 }

 // Update admin if provided
 if let Some(admin) = new_admin {
 bridge_state.admin = admin;
 }

 Ok(())
}

// Handler for setting pause status
pub fn set_pause_status_handler(
 ctx: Context<SetPauseStatus>,
 paused: bool,
) -> Result<()> {
 ctx.accounts.bridge_state.paused = paused;
 Ok(())
}

// Handler for updating the pool ID
pub fn update_pool_id_handler(
 ctx: Context<UpdatePoolId>,
) -> Result<()> {
 ctx.accounts.bridge_state.pool_id = ctx.accounts.new_pool_id.key();
 Ok(())
}

```

Create `programs/bridge/src/instructions/withdraw.rs`:

```

use anchor_lang::prelude::*;
use anchor_spl::token::{self, Token, TokenAccount, Transfer};
use crate::state::*;
use crate::errors::*;

#[derive(Accounts)]
pub struct Withdraw<'info> {
 #[account(mut)]
 pub user: Signer<'info>,

 #[account(
 mut,
 seeds = [b"bridge_state"],
 bump = bridge_state.bump,
)]

```

```

pub bridge_state: Account<'info, BridgeState>,

#[account(
 mut,
 constraint = user_transaction.user == user.key() @
BridgeError::Unauthorized,
 constraint = user_transaction.status == TransactionStatus::Swapped @
BridgeError::InvalidTransactionStatus,
 close = user, // Close account and return rent to user after
completion
)]
pub user_transaction: Account<'info, UserTransaction>,

#[account(
 mut,
 constraint = user_destination_token.mint ==
user_transaction.output_token,
 constraint = user_destination_token.owner == user.key(),
)]
pub user_destination_token: Account<'info, TokenAccount>,

#[account(
 mut,
 seeds = [b"bridge_token_account",
&user_transaction.output_token.to_bytes()],
 bump,
)]
pub bridge_token_account: Account<'info, TokenAccount>,

/// CHECK: PDA used for signing
#[account(
 seeds = [b"bridge_authority"],
 bump,
)]
pub bridge_authority: UncheckedAccount<'info>,

pub token_program: Program<'info, Token>,
}

pub fn handler(ctx: Context<Withdraw>) -> Result<()> {
 // Check if bridge is paused
 if ctx.accounts.bridge_state.paused {
 return Err(BridgeError::BridgePaused.into());
 }
}

```

```

}

let amount_to_transfer =
ctx.accounts.user_transaction.min_output_amount;

// Transfer tokens from bridge to user
let bridge_authority_bump = *ctx.bumps.get("bridge_authority").unwrap();
let bridge_authority_seeds = &[
 b"bridge_authority".as_ref(),
 &[bridge_authority_bump]
];
let signer_seeds = &[&bridge_authority_seeds[..]];

let transfer_ctx = CpiContext::new_with_signer(
 ctx.accounts.token_program.to_account_info(),
 Transfer {
 from: ctx.accounts.bridge_token_account.to_account_info(),
 to: ctx.accounts.user_destination_token.to_account_info(),
 authority: ctx.accounts.bridge_authority.to_account_info(),
 },
 signer_seeds,
);

token::transfer(transfer_ctx, amount_to_transfer)?;

// Update transaction status (not strictly necessary since we're closing
the account)
let user_transaction = &mut ctx.accounts.user_transaction;
user_transaction.status = TransactionStatus::Completed;
user_transaction.completed_at = Some(Clock::get()?.unix_timestamp);

// Update bridge state if output token is JCKP
if user_transaction.output_token == ctx.accounts.bridge_state.jckp_mint
{
 ctx.accounts.bridge_state.total_jckp_processed =
ctx.accounts.bridge_state.total_jckp_processed
 .checked_add(amount_to_transfer)
 .unwrap();
}

Ok(())
}

```

## Security Setup

Security is a critical aspect of any bridge implementation. Let's implement robust security measures to protect user funds.

### Access Control

The bridge uses a multi-layered access control approach:

1. **Admin Role:** Only the designated admin can perform administrative functions
2. **Program-Derived Addresses (PDAs):** PDAs are used to securely manage token custody
3. **Transaction-specific Authorization:** Users can only withdraw their own funds

Create a script to set up proper access control, `scripts/setup-security.ts`:

```
import { Connection, Keypair, PublicKey, SystemProgram, Transaction } from
 '@solana/web3.js';
import { Program, Provider, Wallet } from '@coral-xyz/anchor';
import { readFileSync } from 'fs';
import * as dotenv from 'dotenv';
import { Bridge } from '../target/types/bridge';

// Load environment variables
dotenv.config();

// Configure RPC and wallet
const RPC_URL = process.env.SOLANA_RPC_URL ||
 'https://api.devnet.solana.com';
const WALLET_PATH = process.env.WALLET_PATH ||
 '~/.config/solana/devnet.json';
const BRIDGE_PROGRAM_ID = process.env.BRIDGE_PROGRAM_ID || '';

// Load bridge IDL
const idl = JSON.parse(readFileSync('./target/idl/bridge.json', 'utf-8'));

// Load wallet
function loadWallet(): Keypair {
 const walletData = readFileSync(WALLET_PATH.replace('~', process.env.HOME
 || ''), 'utf-8');
 return Keypair.fromSecretKey(
 Buffer.from(JSON.parse(walletData))
);
}
```



```

async function setupSecurity() {
 console.log('Setting up bridge security...');

 // Initialize connection and wallet
 const connection = new Connection(RPC_URL);
 const wallet = loadWallet();

 // Create provider
 const provider = new Provider(
 connection,
 new Wallet(wallet),
 { commitment: 'confirmed' }
);

 // Initialize program
 const program = new Program<Bridge>(
 idl,
 new PublicKey(BRIDGE_PROGRAM_ID),
 provider
);

 // Get bridge state PDA
 const [bridgeState] = PublicKey.findProgramAddressSync(
 [Buffer.from('bridge_state')],
 program.programId
);

 // Set up multisig or admin-only access
 const adminPubkey = wallet.publicKey.toString();
 console.log(`Setting up admin with pubkey: ${adminPubkey}`);

 try {
 // Check if the bridge is already initialized
 const bridgeStateInfo = await
program.account.bridgeState.fetch(bridgeState);
 console.log('Bridge state already initialized with admin:',
bridgeStateInfo.admin.toString());

 // Update admin role if needed
 // You can add logic here to update the admin to a multisig or another
wallet

 } catch (error) {

```

```

 console.log('Bridge state not initialized yet. Please initialize the
bridge first.');
```

```

 }

 console.log('Security setup completed.');
```

```

}

// Run the script
setupSecurity().catch(console.error);
```

## Fee Management

The bridge collects fees to cover operational costs and to prevent abuse. Let's implement fee management:

```

// In scripts/manage-fees.ts

import { Connection, Keypair, PublicKey } from '@solana/web3.js';
import { Program, Provider, Wallet } from '@coral-xyz/anchor';
import { readFileSync } from 'fs';
import * as dotenv from 'dotenv';
import { Bridge } from '../target/types/bridge';

// Load environment variables and setup
// [similar to setup-security.ts]

async function manageFees() {
 console.log('Managing bridge fees...');

 // Initialize connection, wallet, provider, and program
 // [similar to setup-security.ts]

 // Get bridge state PDA
 const [bridgeState] = PublicKey.findProgramAddressSync(
 [Buffer.from('bridge_state')],
 program.programId
);

 try {
 // Get current fee
 const bridgeStateInfo = await
program.account.bridgeState.fetch(bridgeState);
 console.log(`Current fee: ${bridgeStateInfo.feeBps / 100}%`);
```

```

// Update fee if needed
const newFeeBps = parseInt(process.argv[2]) || bridgeStateInfo.feeBps;

if (newFeeBps !== bridgeStateInfo.feeBps) {
 console.log(`Updating fee from ${bridgeStateInfo.feeBps / 100}% to
 ${newFeeBps / 100}%`);

 await program.methods
 .updateSettings({
 newFeeBps: newFeeBps,
 newAdmin: null,
 })
 .accounts({
 admin: wallet.publicKey,
 bridgeState: bridgeState,
 })
 .rpc();

 console.log(`Fee updated successfully to ${newFeeBps / 100}%`);
} else {
 console.log('No fee change needed');
}
} catch (error) {
 console.error('Error managing fees:', error);
}
}

// Run with: ts-node scripts/manage-fees.ts [new_fee_bps]
// Example: ts-node scripts/manage-fees.ts 50 # Set fee to 0.5%
manageFees().catch(console.error);

```

## Emergency Pause Functionality

Create a script to pause/unpause the bridge in case of emergencies, `scripts/emergency-controls.ts`:

```

import { Connection, Keypair, PublicKey } from '@solana/web3.js';
import { Program, Provider, Wallet } from '@coral-xyz/anchor';
import { readFileSync } from 'fs';
import * as dotenv from 'dotenv';
import { Bridge } from '../target/types/bridge';

// Load environment variables and setup
// [similar to other scripts]

```

```

async function emergencyControls() {
 console.log('Bridge emergency controls...');

 // Initialize client setup
 // [similar to other scripts]

 // Get bridge state PDA
 const [bridgeState] = PublicKey.findProgramAddressSync(
 [Buffer.from('bridge_state')],
 program.programId
);

 try {
 // Get current pause status
 const bridgeStateInfo = await
program.account.bridgeState.fetch(bridgeState);
 console.log(`Current status: ${bridgeStateInfo.paused ? 'PAUSED' :
'ACTIVE'}`);

 // Check command line arguments for requested action
 const action = process.argv[2]?.toLowerCase() || '';

 if (action === 'pause' && !bridgeStateInfo.paused) {
 console.log('Pausing bridge...');

 await program.methods
 .setPauseStatus(true)
 .accounts({
 admin: wallet.publicKey,
 bridgeState: bridgeState,
 })
 .rpc();

 console.log('Bridge paused successfully');
 } else if (action === 'unpause' && bridgeStateInfo.paused) {
 console.log('Unpausing bridge...');

 await program.methods
 .setPauseStatus(false)
 .accounts({
 admin: wallet.publicKey,
 bridgeState: bridgeState,

```

```

 })
 .rpc();

 console.log('Bridge unpaused successfully');
 } else {
 console.log(`No changes made. Use 'pause' or 'unpause' as argument.`);
 }
} catch (error) {
 console.error('Error updating bridge status:', error);
}
}

// Run with: ts-node scripts/emergency-controls.ts [pause|unpause]
emergencyControls().catch(console.error);

```

## Monitoring Setup

Monitoring is crucial for maintaining the health and security of the bridge. Let's implement a comprehensive monitoring solution.

### Creating a Bridge Monitor

Create `scripts/monitor-bridge.ts`:

```

import { Connection, Keypair, PublicKey } from '@solana/web3.js';
import { Program, Provider, Wallet, BN } from '@coral-xyz/anchor';
import { readFileSync } from 'fs';
import * as dotenv from 'dotenv';
import * as fs from 'fs';
import { Bridge } from '../target/types/bridge';

dotenv.config();

// Configuration options
const RPC_URL = process.env.SOLANA_RPC_URL ||
 'https://api.devnet.solana.com';
const WALLET_PATH = process.env.WALLET_PATH ||
 '~/.config/solana/devnet.json';
const BRIDGE_PROGRAM_ID = process.env.BRIDGE_PROGRAM_ID || '';
const LOG_FILE = 'bridge-monitor.log';
const ALERT_THRESHOLD_TX_COUNT = 10; // Alert after this many transactions
const CHECK_INTERVAL_MS = 30000; // Check every 30 seconds

// Alert channels (replace with your actual alerting mechanism)
function sendAlert(message: string) {

```

```

log(`ALERT: ${message}`);
// In a real implementation, you would send alerts via:
// - Email
// - SMS
// - Discord/Slack webhook
// - Pager Duty
console.log(`🚨 ALERT: ${message}`);
}

// Logging function
function log(message: string) {
 const timestamp = new Date().toISOString();
 const logMessage = `[${timestamp}] ${message}\n`;

 // Console output
 console.log(message);

 // File output
 fs.appendFileSync(LOG_FILE, logMessage);
}

async function monitorBridge() {
 log('Starting bridge monitor...');

 // Setup connection and program
 const connection = new Connection(RPC_URL);
 const wallet = loadWallet();
 const provider = new Provider(connection, new Wallet(wallet), {
commitment: 'confirmed' });
 const idl = JSON.parse(readFileSync('./target/idl/bridge.json', 'utf-8'));
 const program = new Program<Bridge>(idl, new PublicKey(BRIDGE_PROGRAM_ID),
provider);

 // Get bridge state PDA
 const [bridgeState] = PublicKey.findProgramAddressSync(
 [Buffer.from('bridge_state')],
 program.programId
);

 // Track processed amounts for change detection
 let lastProcessedJCK = new BN(0);
 let lastProcessedJCKP = new BN(0);

```

```

// Start monitoring loop
log(`Monitoring bridge at ${BRIDGE_PROGRAM_ID}`);

while (true) {
 try {
 // Fetch latest bridge state
 const state = await program.account.bridgeState.fetch(bridgeState);

 // Check if bridge is paused
 if (state.paused) {
 log('⚠ Bridge is currently PAUSED');
 }

 // Check processed amounts and detect changes
 const currentJCK = new BN(state.totalJckProcessed.toString());
 const currentJCKP = new BN(state.totalJckpProcessed.toString());

 if (!currentJCK.eq(lastProcessedJCK)) {
 const diff = currentJCK.sub(lastProcessedJCK);
 log(`JCK processed amount changed: +${diff.toString()} tokens`);
 lastProcessedJCK = currentJCK;
 }

 if (!currentJCKP.eq(lastProcessedJCKP)) {
 const diff = currentJCKP.sub(lastProcessedJCKP);
 log(`JCKP processed amount changed: +${diff.toString()} tokens`);
 lastProcessedJCKP = currentJCKP;
 }

 // Monitor active user transactions
 const userTransactions = await program.account.userTransaction.all();
 log(`Active transactions: ${userTransactions.length}`);

 // Alert if too many pending transactions
 if (userTransactions.length > ALERT_THRESHOLD_TX_COUNT) {
 sendAlert(`High number of pending transactions:
${userTransactions.length}`);
 }

 // Find stuck transactions (older than 1 hour)
 const currentTime = Math.floor(Date.now() / 1000);
 const stuckTransactions = userTransactions.filter(tx => {
 const createdAt = tx.account.createdAt.toNumber();

```

```

 return (currentTime - createdAt) > 3600 &&
 (tx.account.status.deposited || tx.account.status.swapped);
 });

 if (stuckTransactions.length > 0) {
 sendAlert(`Found ${stuckTransactions.length} stuck transactions`);
 stuckTransactions.forEach(tx => {
 log(`Stuck transaction: ${tx.publicKey.toString()}, Status:
${JSON.stringify(tx.account.status)}`);
 });
 }

 // Check Raydium pool liquidity (would require additional API calls to
Raydium)
 // This is just a placeholder for the actual implementation
 log('Pool status: Healthy');

} catch (error) {
 log(`Error monitoring bridge: ${error}`);
 sendAlert(`Bridge monitor error: ${error}`);
}

// Wait before next check
await new Promise(resolve => setTimeout(resolve, CHECK_INTERVAL_MS));
}
}

// Helper to load wallet
function loadWallet(): Keypair {
 const path = WALLET_PATH.replace('~', process.env.HOME || '');
 const walletData = readFileSync(path, 'utf-8');
 return Keypair.fromSecretKey(Buffer.from(JSON.parse(walletData)));
}

// Run as a daemon process
monitorBridge().catch(error => {
 console.error('Fatal error in bridge monitor:', error);
 process.exit(1);
});
});

```

## Setting Up Automated Monitoring

Create a systemd service to run the monitor continuously:



```
/etc/systemd/system/bridge-monitor.service
[Unit]
Description=JCK/JCKP Bridge Monitor
After=network.target

[Service]
User=solana
WorkingDirectory=/home/solana/solana-bridge-program
ExecStart=/usr/bin/ts-node /home/solana/solana-bridge-
program/scripts/monitor-bridge.ts
Restart=always
RestartSec=10
StandardOutput=syslog
StandardError=syslog
SyslogIdentifier=bridge-monitor
Environment=NODE_ENV=production

[Install]
WantedBy=multi-user.target
```

Enable and start the service:

```
sudo systemctl enable bridge-monitor
sudo systemctl start bridge-monitor
sudo systemctl status bridge-monitor
```

## Deployment Checklist

### Pre-Deployment Tasks

#### 1. Environment Setup

- ☐ Development environment configured properly
- ☐ Rust and Solana CLI installed (verified versions)
- ☐ Anchor framework installed
- ☐ RPC endpoint selected with good reliability

#### 2. Token Preparation

- ☐ JCK token created or imported
- ☐ JCKP token created or imported
- ☐ Token accounts created for bridge custody
- ☐ Initial liquidity available for pool creation

#### 3. Code Review

- ☐ Smart contract code reviewed for security issues
- ☐ Input validation confirmed on all user inputs
- ☐ Error handling properly implemented
- ☐ Security features (pause, admin controls) tested

#### 4. Testing

- ☐ Unit tests pass
- ☐ Integration tests pass
- ☐ Security tests pass
- ☐ Edge case scenarios tested
- ☐ Test deployment on devnet successful

## Deployment Steps

### 1. Build the Program

```
Run from project root
anchor build
```

### 2. Generate Program ID

```
solana-keygen new -o target/deploy/bridge-keypair.json
solana address -k target/deploy/bridge-keypair.json
Update Anchor.toml and lib.rs with the new program ID
```

### 3. Deploy to Devnet

```
Set Solana to devnet
solana config set --url https://api.devnet.solana.com

Deploy the program
anchor deploy

Verify deployment
solana program show --programs | grep bridge
```

### 4. Initialize the Bridge

Create a script `scripts/initialize-bridge.ts`:

```
import { Connection, Keypair, PublicKey } from '@solana/web3.js';
import { Program, Provider, Wallet } from '@coral-xyz/anchor';
import { readFileSync } from 'fs';
import * as dotenv from 'dotenv';
import { Bridge } from '../target/types/bridge';
```

```

dotenv.config();

// Configure RPC and wallet
const RPC_URL = process.env.SOLANA_RPC_URL ||
'https://api.devnet.solana.com';
const WALLET_PATH = process.env.WALLET_PATH ||
'~/config/solana/devnet.json';
const BRIDGE_PROGRAM_ID = process.env.BRIDGE_PROGRAM_ID || '';
const JCK_MINT = process.env.JCK_TOKEN_MINT || '';
const JCKP_MINT = process.env.JCKP_TOKEN_MINT || '';
const FEE_BPS = process.env.BRIDGE_FEE_BPS ?
parseInt(process.env.BRIDGE_FEE_BPS) : 30; // Default 0.3%

async function initializeBridge() {
 console.log('Initializing bridge...');

 // Initialize connection and wallet
 const connection = new Connection(RPC_URL);
 const wallet = loadWallet();
 const provider = new Provider(connection, new Wallet(wallet), {
commitment: 'confirmed' });
 const idl = JSON.parse(readFileSync('./target/idl/bridge.json', 'utf-8'));
 const program = new Program<Bridge>(idl, new PublicKey(BRIDGE_PROGRAM_ID),
provider);

 // Derive PDAs
 const [bridgeState] = PublicKey.findProgramAddressSync(
 [Buffer.from('bridge_state')],
 program.programId
);

 const [bridgeAuthority] = PublicKey.findProgramAddressSync(
 [Buffer.from('bridge_authority')],
 program.programId
);

 // Check if bridge is already initialized
 try {
 const existingState = await
program.account.bridgeState.fetch(bridgeState);
 console.log('Bridge already initialized with admin:',
existingState.admin.toString());
 }

```

```

 return;
 } catch (error) {
 console.log('Bridge not initialized yet, proceeding with
initialization...');
 }

 // Initialize the bridge
 try {
 const jckMint = new PublicKey(JCK_MINT);
 const jckpMint = new PublicKey(JCKP_MINT);

 console.log(`Initializing bridge with:`);
 console.log(`- JCK Mint: ${jckMint.toString()}`);
 console.log(`- JCKP Mint: ${jckpMint.toString()}`);
 console.log(`- Fee: ${FEE_BPS / 100}%`);
 console.log(`- Admin: ${wallet.publicKey.toString()}`);

 const tx = await program.methods
 .initialize(FEE_BPS)
 .accounts({
 admin: wallet.publicKey,
 jckMint: jckMint,
 jckpMint: jckpMint,
 bridgeState: bridgeState,
 bridgeAuthority: bridgeAuthority,
 systemProgram: SystemProgram.programId,
 tokenProgram: TOKEN_PROGRAM_ID,
 rent: SYSVAR_RENT_PUBKEY,
 })
 .rpc();

 console.log('Bridge initialized successfully!');
 console.log('Transaction signature:', tx);

 } catch (error) {
 console.error('Error initializing bridge:', error);
 }
}

// Helper to load wallet
function loadWallet(): Keypair {
 const path = WALLET_PATH.replace('~', process.env.HOME || '');
 const walletData = readFileSync(path, 'utf-8');

```

```
 return Keypair.fromSecretKey(Buffer.from(JSON.parse(walletData)));
 }

// Run the script
initializeBridge().catch(console.error);
```

## 5. Create Token Accounts

Create token accounts for the bridge to hold JCK and JCKP tokens:

```
Run script to create token accounts
ts-node scripts/create-token-accounts.ts
```

## 6. Create Raydium Pool

Create the Raydium pool for JCK/JCKP:

```
Run pool creation script
ts-node scripts/create-pool.ts
```

## 7. Update Bridge with Pool ID

After creating the pool, update the bridge with the pool ID:

```
Run pool ID update script
ts-node scripts/update-pool-id.ts [POOL_ID]
```

## Mainnet Deployment

Deploying to mainnet follows a similar process but requires additional security and verification steps.

### 1. Prepare for Mainnet

```
Create a dedicated mainnet deployment wallet
solana-keygen new -o ~/.config/solana/mainnet-bridge-deployer.json
```

### 2. Create a Separate Mainnet Configuration

```
Create .env.mainnet file
cat > .env.mainnet << EOF
SOLANA_CLUSTER=mainnet-beta
SOLANA_RPC_URL=https://api.mainnet-beta.solana.com
WALLET_PATH=$HOME/.config/solana/mainnet-bridge-deployer.json
JCK_TOKEN_MINT=JCKfRiuXGqEocTRYwn4qtjDCM4iMTDwxQk7pXY3CQn
JCKP_TOKEN_MINT=JCKPTyyik8G7TiG84UzxvUzM1oQJXSgWaK4Reya6YrM
ORACLE_UPDATE_AUTHORITY=mainnet_oracle_authority_pubkey
ADMIN_PUBKEY=mainnet_admin_pubkey
BRIDGE_FEE_BPS=30
```

```
RAYDIUM_SWAP_PROGRAM_ID=AMMR8X2tajWmfPL9VA86hnDz7NFboMxFmYKHoinvQJtC
EOF
```

### 3. Deploy to Mainnet

```
Switch to mainnet
solana config set --url https://api.mainnet-beta.solana.com
solana config set --keypair ~/.config/solana/mainnet-bridge-deployer.json

Fund the deployer wallet with sufficient SOL
About 10 SOL is recommended for program deployment

Build and deploy
anchor build
anchor deploy

Verify the program was deployed
solana program show --programs
```

### 4. Run Mainnet Initialization Scripts

```
Initialize the bridge on mainnet
source .env.mainnet
ts-node scripts/initialize-bridge.ts

Create token accounts
ts-node scripts/create-token-accounts.ts

If pool already exists:
ts-node scripts/update-pool-id.ts [EXISTING_POOL_ID]

Or create a new pool:
ts-node scripts/create-pool.ts
```

### Post-Deployment Verification

After deploying the bridge, perform these verification steps:

#### 1. Verify Program Deployment

```
Check that the program is deployed
solana program show $PROGRAM_ID
```

#### 2. Verify Bridge State

```
Run state verification script
ts-node scripts/verify-bridge-state.ts
```

Example `verify-bridge-state.ts` script:

```
import { Connection, PublicKey } from '@solana/web3.js';
import { Program, Provider, Wallet } from '@coral-xyz/anchor';
import { readFileSync } from 'fs';
import * as dotenv from 'dotenv';
import { Bridge } from '../target/types/bridge';

// Load environment variables and setup
// ...

async function verifyBridgeState() {
 console.log('Verifying bridge state...');

 // Initialize client setup
 // ...

 // Get bridge state PDA
 const [bridgeState] = PublicKey.findProgramAddressSync(
 [Buffer.from('bridge_state')],
 program.programId
);

 try {
 // Fetch bridge state
 const state = await program.account.bridgeState.fetch(bridgeState);

 console.log('Bridge State Verification:');
 console.log('-----');
 console.log(`Admin: ${state.admin}`);
 console.log(`Fee: ${state.feeBps / 100}%`);
 console.log(`Paused: ${state.paused ? 'YES ⚠️' : 'NO ✓'}`);
 console.log(`JCK Mint: ${state.jckMint}`);
 console.log(`JCKP Mint: ${state.jckpMint}`);
 console.log(`Pool ID: ${state.poolId}`);
 console.log(`Total JCK Processed: ${state.totalJckProcessed}`);
 console.log(`Total JCKP Processed: ${state.totalJckpProcessed}`);

 // Verify token mints match expected values
 const expectedJCKMint = new PublicKey(process.env.JCK_TOKEN_MINT || '');
 const expectedJCKPMint = new PublicKey(process.env.JCKP_TOKEN_MINT ||
);
```

```

if (!state.jckMint.equals(expectedJCKMint)) {
 console.error('✗ JCK mint does not match expected value!');
 console.error(`Expected: ${expectedJCKMint}`);
 console.error(`Actual: ${state.jckMint}`);
} else {
 console.log('✓ JCK mint verified');
}

if (!state.jckpMint.equals(expectedJCKPMint)) {
 console.error('✗ JCKP mint does not match expected value!');
 console.error(`Expected: ${expectedJCKPMint}`);
 console.error(`Actual: ${state.jckpMint}`);
} else {
 console.log('✓ JCKP mint verified');
}

// Verify pool exists
if (state.poolId.toString() === '11111111111111111111111111111111') {
 console.error('✗ Pool ID not set!');
} else {
 console.log('✓ Pool ID is set');
 // You can add additional verification of the pool here
}

} catch (error) {
 console.error('Error verifying bridge state:', error);
}
}

// Run the script
verifyBridgeState().catch(console.error);

```

### 3. Test Basic Operations

Create a script to test basic bridge operations with small amounts:

```

Run test operations (with small amounts)
ts-node scripts/test-bridge-operations.ts

```

## Maintenance

### Daily Operations Checklist

Create a script `scripts/daily-health-check.ts` that performs these checks:



```

import { Connection, PublicKey } from '@solana/web3.js';
import { Program, Provider, Wallet } from '@coral-xyz/anchor';
import { readFileSync } from 'fs';
import * as dotenv from 'dotenv';
import { Bridge } from '../target/types/bridge';

dotenv.config();

async function dailyHealthCheck() {
 console.log('Performing daily health check...');

 // Initialize client setup
 const connection = new Connection(process.env.SOLANA_RPC_URL ||
'https://api.devnet.solana.com');
 const wallet = loadWallet();
 const provider = new Provider(connection, new Wallet(wallet), {
commitment: 'confirmed' });
 const idl = JSON.parse(readFileSync('./target/idl/bridge.json', 'utf-8'));
 const program = new Program<Bridge>(idl, new
PublicKey(process.env.BRIDGE_PROGRAM_ID || ''), provider);

 try {
 // 1. Check bridge state
 const [bridgeState] = PublicKey.findProgramAddressSync(
 [Buffer.from('bridge_state')],
 program.programId
);
 const state = await program.account.bridgeState.fetch(bridgeState);

 console.log('Bridge Health Check:');
 console.log('-----');
 console.log(`Status: ${state.paused ? '🛑 PAUSED' : '✅ ACTIVE'}`);

 // 2. Check for pending transactions
 const userTransactions = await program.account.userTransaction.all();
 console.log(`Pending Transactions: ${userTransactions.length}`);

 // 3. Check token balances
 const jckTokenAccount = await deriveTokenAccount(program.programId,
state.jckMint);
 const jckpTokenAccount = await deriveTokenAccount(program.programId,
state.jckpMint);
 }
}

```

```

 const jckBalance = await
connection.getTokenAccountBalance(jckTokenAccount);
 const jckpBalance = await
connection.getTokenAccountBalance(jckpTokenAccount);

 console.log(`JCK Balance: ${jckBalance.value.uiAmount}`);
 console.log(`JCKP Balance: ${jckpBalance.value.uiAmount}`);

 // 4. Check Raydium pool status
 console.log(`Pool ID: ${state.poolId}`);
 // In a full implementation, you would query Raydium API for pool health

 // 5. Verify volumes
 console.log(`Total JCK Processed: ${state.totalJckProcessed}`);
 console.log(`Total JCKP Processed: ${state.totalJckpProcessed}`);

 return true;
 } catch (error) {
 console.error('Health check failed:', error);
 return false;
 }
}

// Helper functions
function loadWallet() {
 // Wallet loading code...
}

async function deriveTokenAccount(programId: PublicKey, mint: PublicKey) {
 // Derive token account PDA...
}

// Run the health check
dailyHealthCheck().then(
 success => console.log(`Health check ${success ? 'passed' : 'failed'}`)
).catch(console.error);

```

## Emergency Procedures

Create an emergency response plan for various scenarios:

### Emergency Response Plan

Create a file `docs/emergency-response.md`:

## # Bridge Emergency Response Plan

### ## Emergency Contacts

- Primary Admin: [Name] - [Phone] - [Email]
- Secondary Admin: [Name] - [Phone] - [Email]
- Lead Developer: [Name] - [Phone] - [Email]

### ## Emergency Scenarios and Responses

#### ### 1. Security Breach Detected

##### **\*\*Signs:\*\***

- Unexpected token movements
- Unauthorized admin actions
- Unusual transaction patterns

##### **\*\*Immediate Actions:\*\***

##### 1. Pause the bridge immediately:

```
```bash
ts-node scripts/emergency-controls.ts pause
```

2. Notify all team members
3. Investigate the breach source
4. Document affected transactions

Recovery Steps:

1. Identify vulnerability
2. Develop and test fix
3. Deploy fix (see Update Procedures)
4. Verify security
5. Unpause bridge when confirmed secure

2. Pool Liquidity Crisis

Signs:

- High slippage on swaps
- Low liquidity warnings from monitoring
- Price impact > 5%

Immediate Actions:

1. Pause bridge or increase fees to slow volume:

```
ts-node scripts/manage-fees.ts 100 # Set 1% fee
```

2. Notify liquidity providers
3. Prepare emergency liquidity if available

Recovery Steps:

1. Add liquidity to pool
2. Gradually reduce fees as liquidity stabilizes
3. Monitor price impact closely

3. Oracle Price Deviation

Signs:

- Oracle prices significantly different from market
- Failed transactions due to slippage protection

Immediate Actions:

1. Pause bridge:

```
ts-node scripts/emergency-controls.ts pause
```

2. Manually verify correct prices
3. Update oracle prices:

```
ts-node scripts/update-oracle-prices.ts --jck-price X --jckp-price Y
```

Recovery Steps:

1. Verify price feed accuracy
2. Resume bridge operations:

```
ts-node scripts/emergency-controls.ts unpause
```

Backup Procedures

Create a comprehensive backup strategy to protect critical data:

1. Bridge State Backup

Create `scripts/backup-bridge-state.ts`:

```

```typescript
import { Connection, PublicKey } from '@solana/web3.js';
import { Program, Provider, Wallet } from '@coral-xyz/anchor';
import { readFileSync, writeFileSync } from 'fs';
import * as dotenv from 'dotenv';
import * as path from 'path';
import { Bridge } from '../target/types/bridge';

dotenv.config();

async function backupBridgeState() {
 console.log('Backing up bridge state...');

 // Initialize client setup
 const connection = new Connection(process.env.SOLANA_RPC_URL ||
'https://api.devnet.solana.com');
 const wallet = loadWallet();
 const provider = new Provider(connection, new Wallet(wallet), {
commitment: 'confirmed' });
 const idl = JSON.parse(readFileSync('./target/idl/bridge.json', 'utf-8'));
 const program = new Program<Bridge>(idl, new
PublicKey(process.env.BRIDGE_PROGRAM_ID || ''), provider);

 try {
 // Get bridge state PDA
 const [bridgeState] = PublicKey.findProgramAddressSync(
 [Buffer.from('bridge_state')],
 program.programId
);

 // Fetch bridge state
 const state = await program.account.bridgeState.fetch(bridgeState);

 // Get all user transactions
 const userTransactions = await program.account.userTransaction.all();

 // Create backup data
 const backup = {
 timestamp: new Date().toISOString(),
 network: process.env.SOLANA_CLUSTER,
 bridgeState: {
 address: bridgeState.toString(),
 admin: state.admin.toString(),

```

```

 feeBps: state.feeBps,
 paused: state.paused,
 jckMint: state.jckMint.toString(),
 jckpMint: state.jckpMint.toString(),
 poolId: state.poolId.toString(),
 totalJckProcessed: state.totalJckProcessed.toString(),
 totalJckpProcessed: state.totalJckpProcessed.toString(),
 },
 userTransactions: userTransactions.map(tx => ({
 address: tx.publicKey.toString(),
 user: tx.account.user.toString(),
 inputToken: tx.account.inputToken.toString(),
 inputAmount: tx.account.inputAmount.toString(),
 outputToken: tx.account.outputToken.toString(),
 minOutputAmount: tx.account.minOutputAmount.toString(),
 status: JSON.stringify(tx.account.status),
 createdAt: tx.account.createdAt.toString(),
 completedAt: tx.account.completedAt ?
tx.account.completedAt.toString() : null,
 })),
 });

// Create backup directory if it doesn't exist
const backupDir = path.join(__dirname, '../backups');
if (!fs.existsSync(backupDir)) {
 fs.mkdirSync(backupDir, { recursive: true });
}

// Write backup to file
const backupPath = path.join(
 backupDir,
 `bridge-backup-${process.env.SOLANA_CLUSTER}-${new
Date().toISOString().replace(/[:.]/g, '-')}.json`
);
writeFileSync(backupPath, JSON.stringify(backup, null, 2));

console.log(`Backup saved to ${backupPath}`);
return true;
} catch (error) {
 console.error('Backup failed:', error);
 return false;
}
}

```

```
// Run the backup
backupBridgeState().catch(console.error);
```

## 2. Set up Automated Backups

Add a cron job to run backups daily:

```
Add to crontab
echo "0 0 * * * cd /path/to/solana-bridge-program && ts-node scripts/backup-bridge-state.ts >> logs/backup.log 2>&1" | crontab -
```

## 3. Secure Off-site Backup Storage

Add a script to push backups to secure storage:

```
#!/bin/bash
scripts/upload-backups.sh

BACKUP_DIR="./backups"
REMOTE_BUCKET="s3://your-secure-bucket/bridge-backups"

Encrypt backups before upload
for file in "$BACKUP_DIR"/*.json; do
 if [-f "$file"]; then
 gpg --encrypt --recipient your-gpg-key "$file"
 fi
done

Upload encrypted backups
aws s3 sync "$BACKUP_DIR" "$REMOTE_BUCKET" --exclude "*" --include "*.gpg"

Clean up old backups (keep last 30 days)
find "$BACKUP_DIR" -name "*.json" -mtime +30 -delete
find "$BACKUP_DIR" -name "*.gpg" -mtime +30 -delete
```

## Update Procedures

Create a safe process for updating the bridge program:

### 1. Bridge Program Update Procedure

Create a file `docs/update-procedure.md`:

```
Bridge Update Procedure

Pre-Update Checklist
```

- [ ] Code changes reviewed by at least two developers
- [ ] All tests pass (unit, integration, security)
- [ ] Changes tested on devnet
- [ ] Update announcement made to users (if service interruption expected)
- [ ] Backup of current state completed

## ## Update Steps

### ### 1. Prepare the Update

```
```bash
# Create update branch
git checkout -b update-vX.Y.Z

# Make code changes
# ...

# Run tests
anchor test

# Build program
anchor build
```

2. Test on Devnet

```
# Deploy to devnet
```